

Explanation - Shlok Jaiswal

Reliable Data Transfer over UDP (CS-30003 CA1, P3)

Member: Shlok Jaiswal (24051051) | Part: Channel emulator, experiment harness, plotting/analysis, integrity checks

Your files: shlok_jaiswal/channel_emulator.py, file_hash.py, run_experiments.py, plot_results.py, integrity_check.py

The 1-minute story

- We run everything on one computer, where a network never loses packets. So I built a **fake bad network** (the channel emulator). It sits between sender and receiver and can **drop, duplicate, corrupt, delay and reorder** packets with probabilities we choose.
- It uses a **seeded random generator**: same seed = same random choices = repeatable experiments (needed for fair comparison).
- The **experiment harness** runs all three protocols many times under different conditions and saves numbers (goodput, retransmissions, time) into CSV files. The **plot script** turns them into graphs.
- The **integrity check** uses SHA-256: if the hash of the sent file equals the hash of the received file, the transfer was perfect.

How the emulator works (one packet's journey)

Protocol calls `chan.sendto(packet)` --> roll dice: lost? (then stop) --> roll dice: duplicate? (send two copies) --> for each copy: roll dice: corrupt? (flip one byte) --> delay = base delay + random jitter (+ extra if 'reordered') --> put it in a 'waiting room' and let a background thread send it when the delay is over.

Reordering trick: a 'reordered' packet just gets 20 ms extra delay, so the packets sent after it arrive BEFORE it. Simple and effective.

The three experiments (from our proposal)

- **A: goodput vs loss rate** - loss 0% to 30%, window fixed at 8. Expected: SR > GBN > Stop-and-Wait, gap widening with loss.
- **B: goodput vs window size** - window 1,2,4,8,16 at 10% loss. Bigger window helps GBN and SR; Stop-and-Wait doesn't use a window (flat line).
- **C: retransmissions vs reordering** - reorder probability 0% to 40%, no loss. GBN suffers because it discards out-of-order packets and must resend; SR buffers them.
- Each setting is run 3 times with seeds 1, 2, 3 and averaged. Every run also asserts that the file arrived intact (SHA-256). SR runs with a fixed timeout so the comparison is fair.
- **Goodput** = file size / time taken (useful bytes per second).

How to run

`python shlok_jaiswal/integrity_check.py` (quick test, ~10 s)

`python shlok_jaiswal/run_experiments.py` (all experiments, several minutes; or add a, b or c)

```
python shlok_jaiswal/plot_results.py (makes the PNG graphs in results/)
```

File: shlok_jaiswal/channel_emulator.py

Code	What it means (plain English)
<pre>import heapq import random import threading import time</pre>	heapq = a 'waiting room' that always gives the earliest item first. random = dice rolls. threading = a background thread that sends delayed packets without blocking the sender. time = current clock time.
<pre>REORDER_EXTRA_DELAY = 0.02 # a "reordered" packet is delayed 20 ms extra, so later packets overtake it</pre>	A packet chosen for reordering is delayed 20 ms more than normal, so later packets overtake it.
<pre>class ChannelSocket:</pre>	Our emulator. It wraps a real UDP socket and behaves like one (sendto, recvfrom...), so the protocols don't know they are using a fake network.
<pre> def __init__(self, sock, loss=0.0, duplicate=0.0, reorder=0.0, corrupt=0.0, delay=0.0, jitter=0.0, seed=None):</pre>	Constructor. Each setting is a probability between 0 and 1 (e.g. 0.1 = 10%), except delay/jitter which are in seconds.
<pre> self.sock = sock # the real UDP socket underneath</pre>	The real UDP socket underneath.
<pre> self.loss = loss # probability (0 to 1) a packet is dropped self.duplicate = duplicate # probability a packet is sent twice self.reorder = reorder # probability a packet is delayed extra self.corrupt = corrupt # probability a byte in the packet is damaged</pre>	Save the four probabilities.
<pre> self.delay = delay # fixed one-way delay in seconds self.jitter = jitter # random extra delay between 0 and jitter seconds</pre>	Fixed delay plus random extra delay (jitter).
<pre> self.rng = random.Random(seed) # our own random generator (seeded = repeatable)</pre>	OUR OWN random generator. Created with a seed, it gives the same sequence every time: this is the 'seed-deterministic' requirement.
<pre> self.dropped = 0 # counters for reporting self.corrupted = 0 self.duplicated = 0</pre>	Counters so we can report how many packets were affected.
<pre> self.waiting = [] # the waiting room: (release_time, number, packet, addr)</pre>	The 'waiting room': a list (heap) of delayed packets, each stored with the clock time at which it must be sent.
<pre> self.counter = 0 # tie-breaker so equal release times keep sending order</pre>	A serial number for each queued packet. If two packets have exactly the same release time, this keeps them in their original order.
<pre> self.closed = False</pre>	Set to True when we shut the channel down, so the background thread can stop.
<pre> self.lock = threading.Condition() # lets the worker sleep until something arrives</pre>	A lock plus a 'wake-up bell'. The sending thread and the worker thread both touch self.waiting, so access must be protected; the bell lets the worker sleep until there is something to do.
<pre> threading.Thread(target=self._worker, daemon=True).start()</pre>	Start ONE background worker thread that will deliver the delayed packets. daemon=True means it never keeps the program alive.

Code	What it means (plain English)
<code>def sendto(self, data, addr):</code>	Called by the protocols instead of the normal <code>sendto</code> . Here the network misbehaves.
<code>if self.rng.random() < self.loss: # roll the dice: lose the packet?</code>	<code>random()</code> gives a number between 0 and 1. If it is below the loss probability (e.g. below 0.1 in 10% of cases), the packet is lost.
<code>self.dropped += 1</code> <code>return</code>	Count it and return WITHOUT sending: the packet vanishes.
<code>copies = 1</code>	Normally one copy is sent.
<code>if self.rng.random() < self.duplicate:</code> <code>copies = 2 # send the same packet twice</code> <code>self.duplicated += 1</code>	With the duplicate probability, send two copies of the same packet.
<code>for _ in range(copies):</code>	Repeat for each copy.
<code>packet = data</code>	Start with the untouched bytes.
<code>if self.rng.random() < self.corrupt:</code> <code>packet = self._damage(data)</code> <code>self.corrupted += 1</code>	With the corrupt probability, damage the packet (checksum will catch this on the other side).
<code>wait = self.delay + self.rng.uniform(0, self.jitter)</code>	Delay = fixed delay + a random amount between 0 and jitter.
<code>if self.rng.random() < self.reorder:</code> <code>wait += REORDER_EXTRA_DELAY</code>	With the reorder probability add 20 ms extra, so later packets arrive first = reordering.
<code>self._deliver(packet, addr, wait)</code>	Hand over for actual sending after `wait` seconds.
<code>def _damage(self, data):</code>	Helper that corrupts a packet.
<code>broken = bytearray(data)</code>	bytes can't be modified; bytearray can.
<code>position = self.rng.randrange(len(broken)) # pick a random byte</code>	Pick a random byte position.
<code>broken[position] ^= 0xFF # flip all its bits</code>	XOR with 0xFF flips all 8 bits of that byte, so the value always changes.
<code>return bytes(broken)</code>	Convert back to bytes and return.
<code>def _deliver(self, packet, addr, wait):</code>	Sends the packet now or after `wait` seconds.
<code>if wait <= 0:</code> <code>self._raw_send(packet, addr) # no delay -> send right now</code> <code>return</code>	No delay: send immediately.
<code>with self.lock:</code>	Take the lock so the worker thread doesn't touch the waiting room at the same moment.
<code>heapq.heappush(self.waiting, (time.time() + wait, self.counter, packet, addr))</code> <code>self.counter += 1 # put the packet in the waiting room</code>	Put the packet in the waiting room with its release time = now + wait. The heap keeps the earliest release time at the front.
<code>self.lock.notify() # wake the worker</code>	Ring the bell: wake the worker so it re-checks who is due first.
<code>def _worker(self):</code>	The background thread's job: send each waiting packet when its time comes.

Code	What it means (plain English)
<pre>while True: with self.lock:</pre>	Loop forever; each turn holds the lock while looking at the waiting room.
<pre> if self.closed: return</pre>	Channel closed: end the thread.
<pre> if not self.waiting: self.lock.wait() # nothing to send: sleep continue</pre>	Nothing waiting: sleep until the bell rings, then check again.
<pre> first release, _, packet, addr = self.waiting[0] # the packet due</pre>	Peek at the packet due first (index 0 of the heap) without removing it.
<pre> now = time.time() if release > now: self.lock.wait(release - now) # not due yet: sleep until it is continue</pre>	If it is not due yet, sleep until then (or until the bell rings because a more urgent packet arrived), then re-check.
<pre> heapq.heappop(self.waiting) # due now: take it out</pre>	Its time has come: remove it from the waiting room.
<pre> self._raw_send(packet, addr) # send outside the lock</pre>	Actually send it (outside the lock). Because ONE worker sends packets in release-time order, packets with equal delay always stay in order; only the reorder option can change the order. (Earlier I used one timer thread per packet and found they raced each other, which caused accidental reordering that hurt Go-Back-N.)
<pre>def _raw_send(self, packet, addr):</pre>	The moment the packet really goes into the UDP socket.
<pre> try: self.sock.sendto(packet, addr) except OSError: pass # socket already closed, ignore</pre>	Send it. If the socket was already closed (transfer finished), ignore the error.
<pre>def recvfrom(self, size): return self.sock.recvfrom(size)</pre>	Receiving is passed straight to the real socket (the emulator only affects what is SENT; both directions are covered because both sender and receiver have an emulator for their sends).
<pre>def settimeout(self, seconds): self.sock.settimeout(seconds)</pre>	Pass-through: how long recvfrom may block.
<pre>def close(self): with self.lock: self.closed = True # tell the worker to stop self.lock.notify() self.sock.close()</pre>	Shut down: tell the worker thread to stop (closed = True and wake it), then close the real socket.

File: shlok_jaiswal/file_hash.py

Code	What it means (plain English)
<code>import hashlib</code>	hashlib gives us SHA-256.
<code>def calculate_file_hash(filename):</code>	Returns the SHA-256 fingerprint of a file.
<code>digest = hashlib.sha256() # create an empty SHA-256 calculator</code>	Create an empty SHA-256 calculator.
<code>with open(filename, "rb") as f: # open the file in binary mode</code>	Open the file in binary mode (rb); 'with' closes it automatically.
<code>while True:</code> <code>block = f.read(4096) # read 4 KB at a time</code>	Read the file 4096 bytes at a time (so big files don't fill memory).
<code>if not block: # empty read = end of file</code> <code>break</code>	An empty read means end of file.
<code>digest.update(block) # feed the block to the calculator</code>	Feed this block to the hash calculator.
<code>return digest.hexdigest() # final hash as a hex string</code>	The final 64-character hex fingerprint. Even 1 changed bit gives a totally different hash.
<code>def files_match(first, second):</code> <code>return calculate_file_hash(first) == calculate_file_hash(second)</code>	True if both files have the same hash = same content. This is our end-to-end integrity check.

File: shlok_jaiswal/run_experiments.py

Code	What it means (plain English)
import csv import os import random import sys	csv = save results. os = paths. random = generate the test file. sys = read command-line arguments.
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))	Project top folder.
sys.path.insert(0, os.path.join(ROOT, "main_app"))	Allow importing main.py from the main_app folder.
from main import run_transfer # main.py also sets up the other folders	The function that does ONE full transfer and returns its statistics.
RESULTS = os.path.join(ROOT, "results")	Folder where CSV files and plots are saved.
TEST_FILE = os.path.join(RESULTS, "test_file.bin")	The file we transfer in every experiment.
OUT_FILE = os.path.join(RESULTS, "received_%d.bin" % os.getpid()) # own output file per run	Where the received file goes (name includes the process id so that parallel runs don't overwrite each other).
PROTOCOLS = ["stop-and-wait", "gbn", "sr"]	The three protocols we compare.
RUNS = 3 # repeat each setting 3 times (different seeds)	Repeat every setting 3 times to reduce randomness (proposal: report mean of several runs).
def make_test_file():	Creates the 40 KB test file.
os.makedirs(RESULTS, exist_ok=True)	Create the results folder if missing.
if os.path.exists(TEST_FILE): return	If the file already exists don't rebuild it.
with open(TEST_FILE, "wb") as f: f.write(random.Random(42).randbytes(40000))	Write 40,000 random bytes. Seed 42 makes it identical every time.
def run_one(name, columns, settings_list):	Runs one whole experiment: for every setting, protocol and repeat.
rows = []	List that collects one result row per run.
for x, kwargs in settings_list:	x = the value on the x-axis (loss, window or reorder); kwargs = the settings to pass to run_transfer.
for protocol in PROTOCOLS: for run in range(RUNS):	Loop over the 3 protocols and 3 repeats.
r = run_transfer(protocol, TEST_FILE, OUT_FILE, seed=run + 1, adaptive=False, **kwargs)	Do one transfer. Seed = run + 1 (so runs differ but are repeatable). adaptive=False = fixed timeout for everyone (fair control).
assert r["intact"], "file corrupted!"	Stop immediately if the file arrived damaged: an experiment with a broken transfer would be meaningless.
rows.append([x, protocol, run, round(r["goodput"], 1), r["retransmissions"], round(r["time"], 3)])	Save x-value, protocol, run number, goodput, retransmissions and time.

Code	What it means (plain English)
<pre>print(name, x, protocol, "run", run, "goodput", round(r["goodput"]))</pre>	Progress message on screen.
<pre>with open(os.path.join(RESULTS, name + ".csv"), "w", newline="") as f:</pre>	Open the CSV file for this experiment.
<pre>writer = csv.writer(f)</pre>	Helper that writes CSV rows.
<pre>writer.writerow([columns, "protocol", "run", "goodput", "retransmissions", "time"])</pre>	First row = column names.
<pre>writer.writerows(rows)</pre>	Write all results.
<pre>def experiment_a(): # loss 0% to 30%, window 8</pre>	Experiment A: goodput vs loss.
<pre>run_one("exp_a_loss", "loss", [(l, dict(loss=l, window=8)) for l in [0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30]])</pre>	Loss from 0% to 30% in steps of 5%, window fixed at 8.
<pre>def experiment_b(): # window 1..16, loss 10%</pre>	Experiment B: goodput vs window size.
<pre>run_one("exp_b_window", "window", [(w, dict(loss=0.10, window=w)) for w in [1, 2, 4, 8, 16]])</pre>	Window 1, 2, 4, 8, 16 with loss fixed at 10%.
<pre>def experiment_c(): # reordering 0..40%, no loss, window 8</pre>	Experiment C: retransmissions vs reordering.
<pre>run_one("exp_c_reorder", "reorder", [(p, dict(reorder=p, window=8)) for p in [0, 0.1, 0.2, 0.3, 0.4]])</pre>	Reorder probability 0% to 40%, no loss, window 8.
<pre>if __name__ == "__main__":</pre>	Only when run directly:
<pre>make_test_file()</pre>	Create the test file.
<pre>chosen = sys.argv[1:] or ["a", "b", "c"]</pre>	Which experiments to run: what the user typed after the script name, or all three if nothing.
<pre>if "a" in chosen: experiment_a() if "b" in chosen: experiment_b() if "c" in chosen: experiment_c()</pre>	Run the chosen experiments.
<pre>if os.path.exists(OUT_FILE): os.remove(OUT_FILE) # clean up the temporary received file</pre>	Delete the temporary received file.

File: shlok_jaiswal/plot_results.py

Code	What it means (plain English)
<pre>import os import matplotlib matplotlib.use("Agg") # draw to files, no window needed</pre>	Agg = draw straight into image files (no window), so it works everywhere.
<pre>import matplotlib.pyplot as plt import pandas as pd</pre>	pyplot draws graphs. pandas reads CSV and averages the groups.
<pre>ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) RESULTS = os.path.join(ROOT, "results")</pre>	Project folder and results folder.
<pre>def draw(csv_name, x_column, y_column, title, xlabel, ylabel, png_name):</pre>	One function that draws one graph: which CSV, which column is x and y, the labels and the output file name.
<pre> path = os.path.join(RESULTS, csv_name) if not os.path.exists(path): print("skipping", csv_name, "(run the experiments first)") return</pre>	If the CSV is missing (experiment not run yet) print a message and skip.
<pre> df = pd.read_csv(path)</pre>	Load the CSV into a table (DataFrame).
<pre> mean = df.groupby([x_column, "protocol"])[y_column].mean().reset_index() # average of the runs</pre>	Group rows with the same x-value and protocol (i.e. the 3 repeated runs) and average them.
<pre> plt.figure(figsize=(6, 4))</pre>	New empty graph, 6x4 inches.
<pre> for protocol in ["stop-and-wait", "gbn", "sr"]:</pre>	One line per protocol.
<pre> part = mean[mean["protocol"] == protocol]</pre>	Pick only that protocol's rows.
<pre> plt.plot(part[x_column], part[y_column], marker="o", label=protocol)</pre>	Draw the line with a dot at each point; label is used in the legend.
<pre> plt.title(title) plt.xlabel(xlabel) plt.ylabel(ylabel)</pre>	Title and axis labels.
<pre> plt.legend() plt.grid(True) plt.tight_layout()</pre>	Legend, grid lines and neat spacing.
<pre> plt.savefig(os.path.join(RESULTS, png_name), dpi=120)</pre>	Save the picture as PNG.
<pre> plt.close()</pre>	Free memory.
<pre> print("saved", png_name)</pre>	Confirmation message.
<pre>draw("exp_a_loss.csv", "loss", "goodput", "Goodput vs loss rate (window 8)", "loss probability", "goodput (bytes/s)", "goodput_vs_loss.png")</pre>	Graph 1: goodput vs loss rate.
<pre>draw("exp_b_window.csv", "window", "goodput", "Goodput vs window size (loss 10%)", "window size", "goodput (bytes/s)", "goodput_vs_window.png")</pre>	Graph 2: goodput vs window size.

Code	What it means (plain English)
<pre>draw("exp_c_reorder.csv", "reorder", "retransmissions", "Retransmissions vs reordering (window 8)", "reorder probability", "retransmissions", "retransmissions_vs_reorder.png")</pre>	Graph 3: retransmissions vs reordering.

File: shlok_jaiswal/integrity_check.py

Code	What it means (plain English)
<pre>import os import random import sys</pre>	Standard helpers.
<pre>ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) sys.path.insert(0, os.path.join(ROOT, "main_app")) from main import run_transfer</pre>	Find the project folder and import run_transfer from main.py.
<pre>os.makedirs(os.path.join(ROOT, "results"), exist_ok=True)</pre>	Make sure the results folder exists.
<pre>source = os.path.join(ROOT, "results", "check_in.bin") target = os.path.join(ROOT, "results", "check_out.bin")</pre>	Names of the file to send and the received copy.
<pre>with open(source, "wb") as f: f.write(random.Random(7).randbytes(15000))</pre>	Create a 15 KB random test file.
<pre>for protocol in ["stop-and-wait", "gbn", "sr"]:</pre>	Test all three protocols.
<pre> r = run_transfer(protocol, source, target, window=8, loss=0.15, duplicate=0.1, reorder=0.1, corrupt=0.1, seed=3)</pre>	Transfer under a really bad network: 15% loss, 10% duplicates, 10% reordering and 10% corruption together.
<pre> status = "PASS" if r["intact"] else "FAIL"</pre>	Was the file identical (SHA-256 match)?
<pre> print(status, protocol, "retransmissions =", r["retransmissions"], "time = %.2fs" % r["time"])</pre>	Show the result line.
<pre> if not r["intact"]: sys.exit(1)</pre>	Stop with an error if any protocol failed.
<pre>print("All protocols delivered the file intact.")</pre>	Reached only if all three passed.

Viva questions you may be asked (with easy answers)

Q. What does the channel emulator do?

A. It sits between sender and receiver and, with configurable probabilities, drops, duplicates, corrupts, delays and reorders packets. It lets us test reliability on a single machine.

Q. How do you make experiments reproducible?

A. The emulator has its own `random.Random(seed)`. The same seed gives the same sequence of dice rolls, so the same loss pattern. Each setting is also run 3 times and averaged.

Q. How is reordering produced?

A. A chosen packet gets an extra 20 ms delay, so packets sent after it arrive before it. Jitter also causes small natural reordering.

Q. How is corruption produced and detected?

A. One random byte is XOR-ed with 0xFF (all bits flipped). The receiver recomputes the checksum, it doesn't match, and the packet is discarded as if lost.

Q. Why a background thread and waiting room for delay?

A. `time.sleep` would block the sender for the whole delay. A worker thread sends each packet when its time comes, like a real network where packets are 'in flight'. One worker (instead of one timer per packet) keeps equal-delay packets in order; I first used a Timer per packet and saw accidental reordering that unfairly hurt Go-Back-N.

Q. What is goodput vs throughput?

A. Goodput counts only useful file bytes per second (file size / time). Throughput counts all bytes on the wire including retransmissions and headers.

Q. What does your data show?

A. Stop-and-Wait is always lowest (about 47 KB/s with no loss, 8 KB/s at 30% loss). With no loss GBN and SR are equal (about 320 KB/s). As loss rises SR pulls ahead of GBN (at 30% loss about 28 KB/s vs 20 KB/s). With window 16 at 10% loss SR is about 110 KB/s vs GBN 79 KB/s. Honest note: at very small windows (2-4) GBN was slightly better or equal, because with few packets in flight there is little to save; SR's advantage appears at larger windows. In the reordering test GBN's retransmissions rise from 0 to about 93 while SR and Stop-and-Wait stay at 0. Numbers are in `results/*.csv` and the PNG plots.

Q. Why fixed RTO in the experiments?

A. So only the protocol differs. The proposal says RTO tuning is a control unless it is the variable under test.

Q. Why 3 runs per point?

A. Timing has noise. Averaging repeated runs gives a more trustworthy value. (The raw runs are in the CSVs if you want the variance.)

Q. How does the integrity check work?

A. SHA-256 of the original and the received file are compared. Equal hashes mean identical content. run_experiments asserts this on every single run.

Q. What is a seed?

A. The starting value of a random number generator. Same seed = same 'random' numbers.

Q. Why does the emulator wrap sendto but only pass through recvfrom?

A. Everything that goes on the network must pass the emulator; if both sides wrap their sends, DATA and ACKs are both affected. Receiving is simply reading what arrived.